

Trabalho Prático – Problema do Caixeiro Viajante

INSTRUÇÕES GERAIS

1. Este trabalho vale **5 pontos** e é parte da nota da Avaliação Parcial 3. O trabalho poderá ser realizado **em equipes de até 3 (três) integrantes**.
2. A entrega será feita **exclusivamente via SIGAA**, em arquivo .zip, contendo: código-fonte completo; relatório técnico em formato PDF; instruções claras de compilação e execução; *script* utilizado para gerar dados e medir tempos.
3. **Não serão aceitas cópias** de trabalhos. Trabalhos que apresentarem fortes evidências de plágio (entre alunos ou de ferramentas de IA generativa) receberão **nota zero**.
4. Trabalhos enviados fora do prazo, ou por outros meios que não o SIGAA, **não serão aceitos**.

1. O PROBLEMA DO CAIXEIRO VIAJANTE

A definição formal do **Problema do Caixeiro Viajante** (do inglês, *traveling salesman problem* – **TSP**) é dada como segue:

“Dado um grafo $G = (V, E)$ e uma função de custo nas arestas $c: E \rightarrow \mathbb{R}^+$, determinar um ciclo que visite cada vértice (exatamente uma vez) tal que a soma dos pesos das arestas que o compõem seja o mínimo possível.”

Em outras palavras, o TSP consiste em determinar um ciclo hamiltoniano de custo mínimo em um grafo ponderado, o qual pode ser direcionado ou não. O TSP é um problema amplamente estudado nas áreas de Teoria dos Grafos e de Ciência da Computação, em especial sob a ótica da Pesquisa Operacional, Otimização Combinatória e Teoria da Computação. Destaca-se que o problema é de difícil solução, mesmo se todos os pesos das arestas pertencem ao conjunto $\{1, 2\}$. De fato, determinar um ciclo hamiltoniano de custo mínimo em um grafo sem propriedades particulares é **NP-difícil** e não se conhece um algoritmo de aproximação com razão constante para tal.

2. OBJETIVO DO TRABALHO

O objetivo deste trabalho é implementar e avaliar algoritmos para o **TSP**, considerando instâncias geométricas descritas no formato da *Travelling Salesman Problem Library* - **TSPLIB**. Cada equipe deverá desenvolver um método capaz de construir um *tour* que visite todas as cidades exatamente uma vez e retorne à cidade inicial, buscando um bom equilíbrio entre qualidade das soluções obtidas e tempo de processamento/eficiência algorítmica. O trabalho envolve:

- Implementação;
- Avaliação experimental;
- Elaboração de relatório;
- Apresentação oral;
- Participação em competição final.

3. REQUISITOS GERAIS

Cada equipe deverá implementar um método próprio para resolver instâncias do TSP. O método pode ser heurístico, aproximativo ou exato, desde que computacionalmente viável. Não há restrições quanto a linguagem de programação ou paradigma algorítmico adotado. Quanto ao uso de heurísticas básicas, destaca-se que a heurística do Vizinho Mais Próximo pode ser utilizada como *baseline* ou inicialização do método, mas não pode ser utilizada para constituir a solução final isolada. Caso o método implementado trate de heurísticas básicas, deve-se incorporar alguma forma adicional de processamento que vá além.

Não é permitido utilizar bibliotecas prontas específicas para grafos, TSP ou otimização combinatória. É permitido utilizar:

- Bibliotecas padrão da linguagem;
- Bibliotecas com estruturas de dados auxiliares, como: listas, vetores, filas, pilhas, árvores balanceadas e estruturas de apoio equivalentes – desde que seu uso seja tecnicamente justificado e claramente documentado.

O programa implementado deverá receber instâncias no formato da **TSPLIB** – biblioteca com instâncias do TSP e problemas correlatos. Devem ser consideradas apenas instâncias do TSP simétrico. Neste tipo de instância, a distância d_{ij} do vértice i ao vértice j é obrigatoriamente igual à distância d_{ji} do vértice j ao vértice i . Serão consideradas instâncias com coordenadas em 2 dimensões. O arquivo de entrada de um caso de teste, com extensão **.tsp**, consiste em duas partes: a especificação do problema e o conjunto de dados dos pontos.

Na especificação, podem aparecer as seguintes linhas:

- **NAME:** nome da instância;
- **TYPE:** tipo da instância (considerar apenas do tipo TSP);
- **COMMENT:** comentários;
- **DIMENSION:** número de cidades;
- **EDGE_WEIGHT_TYPE:** tipo de distância (considerar apenas do tipo EUC_2D).

Para instâncias **EUC_2D**, a distância entre duas cidades deve ser calculada como:

$$d_{ij} = \text{floor} \left(0,5 + \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \right),$$

em que $\text{floor}(x)$ corresponde à função que retorna o maior número inteiro menor ou igual a x . Em outras palavras, a distância é definida pelo número inteiro mais próximo do valor da distância euclidiana entre i e j . A seção de dados será precedida pelo termo **NODE_COORD_SECTION**. Segue abaixo um exemplo válido:

```
NAME: tulio5
COMMENT: instancia pequena de teste
TYPE: TSP
DIMENSION: 5
EDGE_WEIGHT_TYPE: EUC_2D
NODE_COORD_SECTION
1 288 149
2 288 129
3 270 133
4 256 141
5 256 157
EOF
```

Algumas instâncias adicionais, bem como a documentação do formato, podem ser encontradas em: https://drive.google.com/drive/folders/1nzzl_3npkwNw9depaopideW4D-aGcsId?usp=sharing

O programa deverá produzir a saída no seguinte formato:

```
NAME: tulio5
COMMENT: solucao obtida pelo professor utilizando o metodo xxxxx
TYPE: TOUR
DIMENSION: 5
TOTAL_WEIGHT: 103
```

```
TOUR_SECTION
```

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

```
EOF
```

A saída deve conter obrigatoriamente:

- **NAME:** nome da instância;
- **COMMENT:** nomes dos alunos e método utilizado;
- **TYPE:** sempre TOUR;
- **DIMENSION:** número de cidades;
- **TOTAL_WEIGHT:** custo total do tour;
- **TOUR_SECTION:** ordem das cidades visitadas;
- **EOF:** marca final do arquivo.

É fundamental seguir rigorosamente os nomes dos campos, a formatação especificada, espaços em branco e a ordem dos elementos.

O programa deverá receber os dados da instância por meio da **entrada padrão (stdin)** e produzir a solução por meio da **saída padrão (stdout)**. A execução deverá seguir o formato:

```
./programa < instancia.tsp > instancia.tsp.tour
```

Em que:

- **instancia.tsp** é o arquivo contendo os dados da instância no formato especificado;
- **instancia.tsp.tour** é o arquivo contendo a solução produzida pelo programa.

Toda a leitura dos dados deverá ser feita diretamente da entrada padrão, e toda a saída deverá ser escrita na saída padrão. O programa não deve depender de nomes fixos de arquivos para entrada ou saída. Destaca-se que o programa deve produzir apenas a saída especificada para o tour. Mensagens adicionais (como logs, mensagens de depuração ou avisos) não devem ser enviadas para a saída padrão, pois podem invalidar o formato da solução. Além disso, o programa deve encerrar corretamente após produzir a saída, sem aguardar entrada adicional.

4. AVALIAÇÃO EXPERIMENTAL

Os algoritmos devem ser avaliados em instâncias com tamanhos variados, por exemplo: 50, 100, 200, 500 cidades. Os alunos devem demonstrar que seus algoritmos funcionam adequadamente em diferentes escalas. Cada equipe deverá realizar experimentos próprios e analisar:

- Qualidade das soluções;
- Tempo de execução;
- Comportamento com o aumento do tamanho da instância;
- Impacto de parâmetros (quando existirem).

A análise experimental é parte essencial do trabalho.

5. RELATÓRIO

Cada grupo deverá entregar um relatório que deve conter os seguintes elementos:

- Capa;
- Sumário;
- Introdução: contendo descrição do problema, motivação e objetivos do trabalho;
- Método Proposto: abrangendo descrição detalhada do método implementado, pseudocódigo, análise conceitual e justificas das decisões adotadas;
- Resultados e Discussões: apresentar as instâncias utilizadas, metodologia de experimentação, resultados obtidos, análise de pontos fortes e limitações, bem como possíveis melhorias;
- Conclusão.

A clareza da análise é **tão importante quanto os resultados obtidos**.

O relatório deve ser redigido conforme a norma culta da língua portuguesa, com atenção à correção gramatical, clareza e organização textual. Além disso, deve estar adequadamente formatado segundo as convenções de um trabalho acadêmico, incluindo:

- título, identificação dos autores e seções bem definidas;
- paginação;
- figuras e tabelas devidamente numeradas e legendadas;
- referências quando necessárias;
- padronização de fonte, margens e espaçamento.

Trabalhos que não observarem esses critérios poderão ter sua nota reduzida.

6. APRESENTAÇÃO E ARGUIÇÃO

Cada equipe deverá realizar uma apresentação oral com duração estimada de 5 minutos, apresentado a ideia principal do algoritmo, decisões de projeto e principais resultados. Será realizada uma arguição técnica sobre o trabalho, envolvendo o funcionamento do algoritmo, decisões de implementação e resultados experimentais. A apresentação/arguição pode ser utilizada **para validar autoria**.

7. AVALIAÇÃO DO TRABALHO

A nota do trabalho será composta pelos seguintes critérios:

- **Implementação**
- **Relatório**
- **Apresentação**
- **Qualidade da abordagem algorítmica**

Serão considerados aspectos como:

- Correção;
- Eficiência;
- Clareza;
- Originalidade.

7.1. COMPETIÇÃO FINAL

Para avaliar a qualidade algorítmica, será realizada uma **competição entre as equipes**. Cada algoritmo será executado nas mesmas instâncias e nas mesmas condições computacionais. As instâncias utilizadas na competição poderão ser diferentes das instâncias disponibilizadas previamente. Cada instância será executada uma única vez com semente fixa definida pelo professor. Cada execução terá um **tempo máximo**, definido para cada instância. O tempo será medido externamente pelo professor durante a competição. Caso o limite seja excedido, a solução poderá ser desconsiderada ou receber penalização máxima.

Para cada instância i , são definidos:

- C_i : custo da solução obtida pela equipe para a instância i ;
- C_i^{best} : melhor solução encontrada;
- T_i : tempo obtido pela equipe;
- T_i^{best} : melhor tempo observado.

A cada equipe, ao resolver uma dada instância i , serão atribuídos os seguintes valores:

- Q_i : qualidade relativa, calculada por $Q_i = C_i / C_i^{best}$;
- R_i : tempo relativo, calculada por $R_i = T_i / T_i^{best}$;
- $score_i$: pontuação obtida, calculada por $score_i = 0,8 \cdot Q_i + 0,2 \cdot R_i$.

A pontuação final da equipe será dada por $score = \sum_i score_i$. As equipes serão classificadas em ordem crescente de *score* final (quanto menor, melhor).

7.2.DESTAQUES E PREMIAÇÕES

Serão concedidos destaques para:

- Melhores desempenhos gerais: até 1,0 ponto extra para o primeiro colocado, até 0,5 ponto extra para o segundo colocado e até 0,3 ponto extra para o terceiro colocado;
- Abordagem mais interessante: caso não tenha alcançado a primeira colocação, pode ser atribuído até mais 0,5 ponto extra.

A pontuação extra se dará em uma unidade de escolha de cada integrante das equipes contempladas.

8. USO DE FERRAMENTAS DE IA

Ferramentas de IA generativa podem ser utilizadas como **apoio**, desde que seu uso seja declarado explicitamente. Seguindo a Portaria Nº 39/PRPPG/UFC, de 01 de outubro de 2025, é vedado o uso de IA para:

- I. Gerar conteúdo original, interpretações ou análises críticas;
- II. Redigir seções substantivas do trabalho;
- III. Fabricar, alterar, manipular ou “embelezar” dados, resultados, imagens ou gráficos;
- IV. Inserir referências não verificadas ou mascarar plágio.

O relatório deve incluir uma seção intitulada: “Uso de ferramentas de IA”. Nessa seção devem ser informados:

- Quais ferramentas foram utilizadas;
- Para quais finalidades;
- *Prompts* utilizados (quando relevante).

A omissão do uso de tais ferramentas pode ser considerada má conduta acadêmica. Trabalhos que não observarem esses critérios poderão ter sua nota reduzida.